PROJECT NAME:

Virtual Whiteboard

Software Requirements Specification

Course code: INT 221
Course Name: MVC Programming

Student Names & Registration Numbers

1. Shubham Shandilya (12313394)
2. Prashant (12312739)

Table of Contents

1. Introduction

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for the Virtual Whiteboard web application, developed using PHP Laravel as the backend framework. This document serves as a reference for developers, testers, and stakeholders involved in the development of the project.

1.1 Purpose

The purpose of this SRS is to provide a detailed and comprehensive description of the Virtual Whiteboard application. It is intended for the development team, course instructor, and evaluators. The document defines what the system will do, how it will perform, and the constraints under which it must operate.

1.2 Scope

The Virtual Whiteboard is a real-time, browser-based collaborative drawing and annotation platform built with PHP Laravel. The system allows multiple users to join shared whiteboard sessions, draw freely, add text, insert shapes, and collaborate in real time. Key features include:

- Real-time multi-user drawing canvas using WebSockets (Laravel Echo + Pusher)
- User authentication and session management
- Creation, saving, and loading of whiteboard sessions
- Drawing tools: pen, eraser, shapes, text, color picker
- Board sharing via unique shareable links
- Export whiteboard as PNG/PDF
- Responsive UI accessible via desktop and tablet browsers

The application does not include native mobile apps or offline functionality in its initial release.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
Laravel	PHP web application framework following MVC architecture
WebSocket	Full-duplex communication protocol for real-time data transfer
MVC	Model-View-Controller software architectural pattern
API	Application Programming Interface
CRUD	Create, Read, Update, Delete operations
UI	User Interface
DB	Database (MySQL used in this project)
JWT	JSON Web Token for authentication
DFD	Data Flow Diagram

Term	Definition
Canvas	HTML5 element used for drawing graphics via JavaScript

1.4 References

- Laravel Official Documentation — https://laravel.com/docs
- Pusher Documentation — https://pusher.com/docs
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- HTML5 Canvas API — https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API
- Bootstrap 5 Documentation — https://getbootstrap.com/docs/5.0

1.5 Overview

The remainder of this SRS is organized as follows:

- Section 2 provides a general description of the product including its perspective, functions, user characteristics, constraints, and assumptions.
- Section 3 defines specific functional and non-functional requirements.
- Section 4 presents analysis models including Data Flow Diagrams (DFD).
- Sections 5 and 6 contain supporting documentation including the GitHub link and deployment link.

2. General Description

This section describes the general factors that affect the product and its requirements.

2.1 Product Perspective

The Virtual Whiteboard is a standalone web application that operates within a standard web browser. It interacts with a Laravel backend via RESTful APIs and real-time WebSocket connections (via Pusher/Laravel Echo). It uses a MySQL database to persist whiteboard sessions, user data, and canvas states. The system is self-contained but can integrate with third-party authentication (OAuth) in future versions.

The application is organized following the MVC (Model-View-Controller) architecture provided by Laravel, with Blade templates for server-rendered views and a JavaScript-powered frontend canvas.

2.2 Product Functions

The major functions of the Virtual Whiteboard include:

- User Registration and Login — Secure account creation and authentication using Laravel Breeze/Sanctum
- Dashboard — Users can view, create, and manage their whiteboards
- Whiteboard Canvas — A full-screen HTML5 canvas with drawing tools

- Real-Time Collaboration — Multiple users on the same board see each other's drawing in real time
- Tool Panel — Pen, eraser, shapes (rectangle, circle, line), text insertion, color picker, stroke width
- Session Persistence — Canvas state is saved to the database and reloaded on revisit
- Board Sharing — Generate a shareable URL; set boards as public or private
- Export — Download the canvas as a PNG image or PDF document

2.3 User Characteristics

The primary users of the Virtual Whiteboard are:

- Students and Educators — Use the board for collaborative brainstorming, teaching, and note-taking. Basic computer literacy assumed.
- Remote Teams — Professionals using the board for ideation and planning. Familiar with web-based tools.
- General Public — Casual users with minimal technical background who need an easy drawing tool.

All users interact through a web browser and are expected to have a basic understanding of drawing applications.

2.4 General Constraints

- The application must be deployed on a PHP 8.1+ compatible server
- The application requires an active internet connection for real-time features
- Pusher free tier limits apply to concurrent connections in development
- The canvas rendering is dependent on HTML5 Canvas API support in the browser (IE not supported)
- All user-uploaded data must comply with applicable data protection regulations

2.5 Assumptions and Dependencies

- It is assumed that users will access the application via modern browsers (Chrome, Firefox, Edge, Safari)
- The project depends on the Pusher service for WebSocket broadcasting; any downtime of Pusher affects real-time features
- MySQL 8.0 or higher is assumed to be available on the deployment server
- Composer (PHP dependency manager) is available for dependency management
- Node.js and NPM are required for compiling frontend assets (Laravel Mix/Vite)

3. Specific Requirements

This section provides detailed requirements for the Virtual Whiteboard system. Each requirement is correct, traceable, unambiguous, verifiable, prioritized, complete, consistent, and uniquely identifiable.

3.1 External Interface Requirements

3.1.1 User Interfaces

The application shall provide the following user interface screens:

- Landing Page — Introduction to the product with Login/Register buttons
- Registration Form — Fields: Name, Email, Password, Confirm Password
- Login Form — Fields: Email, Password with 'Remember Me' option
- Dashboard — List of user's whiteboards with options to create, open, share, or delete
- Whiteboard Canvas Page — Full-browser canvas with a fixed tool panel on the left
- Settings Page — Account settings, password update, profile management

All UI components shall be built using Bootstrap 5 and shall be responsive for desktop and tablet viewports.

3.1.2 Hardware Interfaces

The application operates on any device with a web browser and a pointer input device (mouse, stylus, or touch). No specific hardware interfaces are required beyond a network interface for internet connectivity.

3.1.3 Software Interfaces

Software	Purpose	Version
PHP / Laravel	Backend framework	PHP 8.1+ / Laravel 10.x
MySQL	Relational database for data persistence	8.0+
Pusher	WebSocket service for real-time collaboration	Pusher JS 7.x
Laravel Echo	Frontend WebSocket listener	1.x
Bootstrap 5	Responsive UI framework	5.x
Vite / Laravel Mix	Asset bundling	Latest
Fabric.js / Konva.js	Canvas drawing library	Latest stable

3.1.4 Communications Interfaces

- HTTPS — All data transmission between client and server shall be encrypted via TLS/SSL
- WebSockets (WSS) — Real-time canvas event broadcasting via Pusher over secure WebSocket
- RESTful API — AJAX calls using Axios for CRUD operations on boards, sessions, and user data

3.2 Functional Requirements

The following subsections describe the functional requirements of the system:

3.2.1 User Authentication

3.2.1.1 Introduction

The system shall provide secure user registration and login.

3.2.1.2 Inputs

- Registration: Name, Email, Password, Password Confirmation
- Login: Email, Password

3.2.1.3 Processing

- Registration: validate inputs, hash password using bcrypt, create user record, send verification email
- Login: validate credentials, issue session token/cookie, redirect to dashboard

3.2.1.4 Outputs

- Successful registration: user account created, redirect to email verification page
- Successful login: authenticated session established, redirect to dashboard

3.2.1.5 Error Handling

- Duplicate email on registration: display 'Email already registered' error
- Invalid credentials on login: display 'Invalid email or password' error
- Validation failures: display field-level error messages

3.2.2 Whiteboard Management (CRUD)

3.2.2.1 Introduction

Authenticated users shall be able to create, view, update, and delete whiteboard sessions.

3.2.2.2 Inputs

- Create: board name, privacy setting (public/private)
- Update: rename board, change privacy
- Delete: board ID confirmation

3.2.2.3 Processing

- Create: generate unique board ID and shareable URL, save initial empty canvas state
- Load: retrieve stored canvas state from DB and render on canvas
- Auto-save: debounce canvas changes and persist JSON state every 10 seconds

3.2.2.4 Outputs

- New board created and opened in canvas editor
- Board list updated on dashboard

3.2.2.5 Error Handling

- Board not found: 404 error page
- Unauthorized access to private board: 403 error page

3.2.3 Real-Time Collaborative Drawing

3.2.3.1 Introduction

Multiple users on the same board shall see each other's drawing actions in real time.

3.2.3.2 Inputs

- User drawing actions: mouse/touch events on canvas (start, move, end)
- Selected tool and color settings

3.2.3.3 Processing

- Each drawing event is broadcast via Laravel Echo to the board's Pusher channel
- All subscribed users receive the event and render it on their canvas

3.2.3.4 Outputs

- All connected users see the drawing in near real-time (< 200ms latency)
- User presence indicators show who is connected to the board

3.2.3.5 Error Handling

- WebSocket disconnection: fall back to polling with user notification
- Conflicting draw events: last-write-wins strategy

3.2.4 Drawing Tools

3.2.4.1 Introduction

The canvas shall provide the following tools:

- Pen — Freehand drawing with adjustable stroke width and color
- Eraser — Remove drawings from canvas
- Shapes — Draw rectangles, circles, triangles, and straight lines
- Text — Add text annotations with font size and color options
- Color Picker — Select stroke/fill color from a color palette
- Undo/Redo — Revert or reapply the last drawing actions
- Clear Board — Remove all drawings from the canvas

3.2.5 Export and Sharing

3.2.5.1 Introduction

Users shall be able to share and export their whiteboards.

3.2.5.2 Inputs

- Share: set board visibility to public; copy shareable URL
- Export: choose format (PNG or PDF)

3.2.5.3 Processing

- Sharing: generate a unique URL with board hash; allow unauthenticated read access for public boards
- Export PNG: use HTML5 Canvas toDataURL() to generate image
- Export PDF: use jsPDF library to convert canvas to PDF

3.2.5.4 Outputs

- Shareable link copied to clipboard with confirmation toast
- PNG/PDF file downloaded to user's browser

3.5 Non-Functional Requirements

3.5.1 Performance

- The application shall load the dashboard within 2 seconds on a standard broadband connection
- Canvas drawing events shall be broadcast to all collaborators within 200 milliseconds
- The system shall support at least 50 concurrent users per whiteboard session
- Auto-save operations shall not block or lag the canvas for more than 100ms

3.5.2 Reliability

- The system shall maintain 99% uptime excluding scheduled maintenance
- Unsaved whiteboard data shall be preserved for up to 60 minutes in the event of a network interruption
- The system shall automatically reconnect WebSocket connections after a drop

3.5.3 Availability

- The application shall be available 24/7
- Planned maintenance windows shall not exceed 1 hour per week and shall be scheduled during off-peak hours

3.5.4 Security

- All passwords shall be hashed using bcrypt before storage
- The application shall protect against SQL Injection, XSS, and CSRF attacks using Laravel's built-in protections
- All communication shall be encrypted via HTTPS/WSS
- Private boards shall only be accessible to their owner and explicitly invited users
- Session tokens shall expire after 24 hours of inactivity

3.5.5 Maintainability

- The codebase shall follow PSR-12 coding standards for PHP
- All major modules shall be unit tested with a minimum of 70% code coverage
- The system shall use Laravel migrations for all database schema changes
- The application shall be documented with PHPDoc comments for all controller methods

3.5.6 Portability

- The application shall run on any PHP 8.1+ compatible Linux server (Ubuntu 20.04+)
- The frontend shall be compatible with Chrome 100+, Firefox 100+, Edge 100+, and Safari 15+
- The application shall support Docker-based deployment using Laravel Sail

3.7 Design Constraints

- The application must use the Laravel PHP framework as the backend (course requirement)
- The database must be MySQL relational database

- Real-time features must use Pusher or a compatible Laravel Echo broadcasting driver
- The UI must use Bootstrap 5 for responsive layout
- No native mobile application is required; browser-based access only

3.9 Other Requirements

- The application must comply with GDPR principles; no user data shall be sold to third parties
- The system shall provide a privacy policy and terms of service page
- User email addresses shall be verified before granting full access to the application

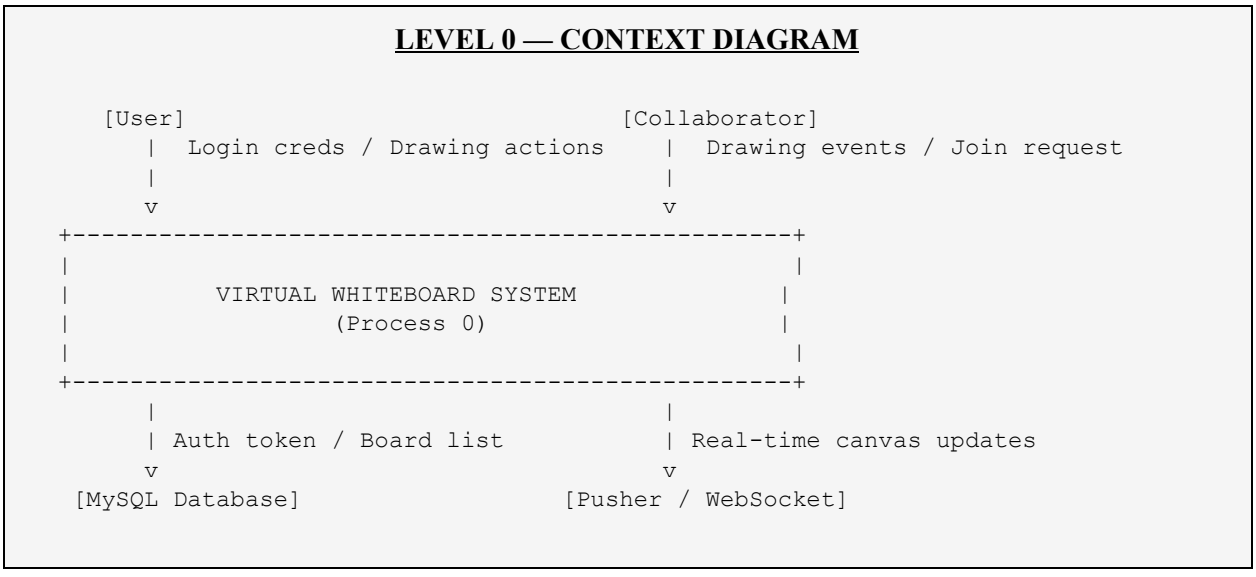
4. Analysis Models

This section presents the Data Flow Diagrams (DFDs) modelling data movement within the Virtual Whiteboard system.

4.1 Data Flow Diagrams (DFD)

Level 0 DFD — Context Diagram

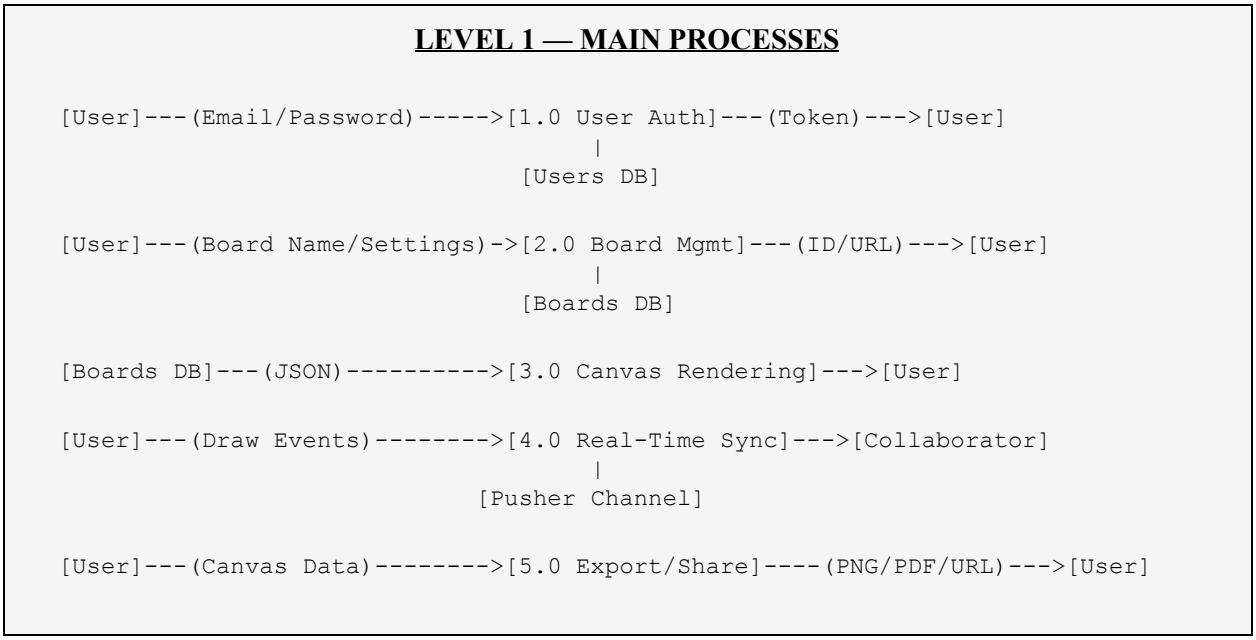
The context diagram represents the entire system as a single process interacting with external entities:



Entity	Data Flow To System	Data Flow From System
User	Login credentials, Drawing actions, Board settings	Auth token, Canvas state, Board list
Collaborator	Drawing events, Join request	Live canvas updates, Presence info
Pusher (WebSocket)	Broadcast events	Real-time drawing events
MySQL Database	Query results	CRUD requests

Level 1 DFD — Main Processes

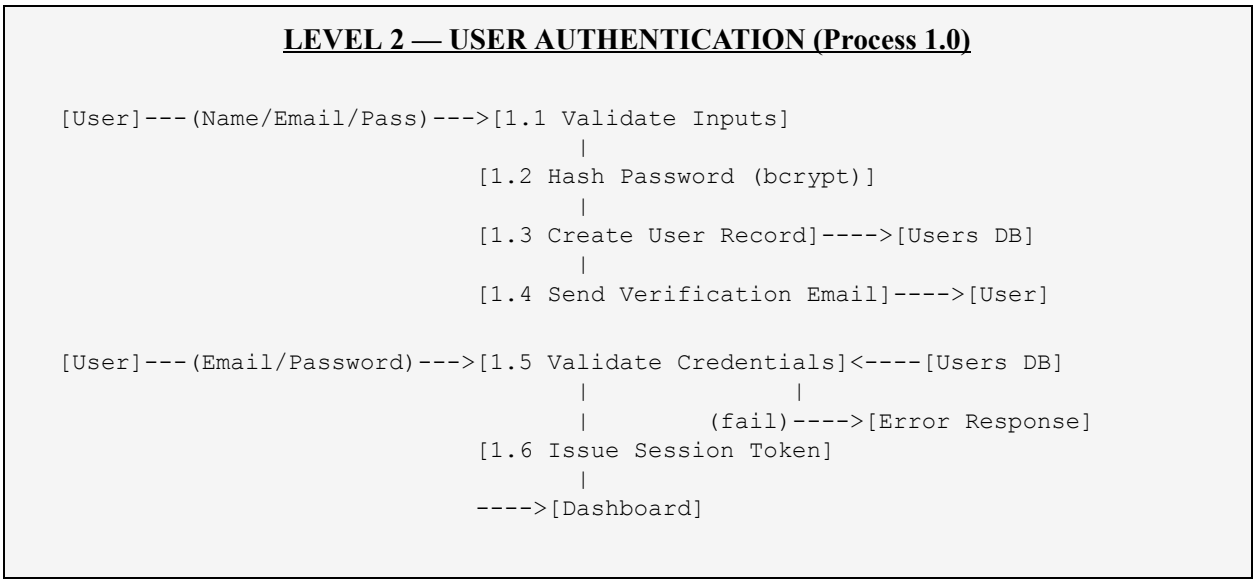
Level 1 expands the context diagram into five main sub-processes:



Process	Input	Output	Data Store
1.0 User Auth	Email, Password	Session Token	Users DB
2.0 Board Management	Board Name, Settings	Board ID, URL	Boards DB
3.0 Canvas Rendering	Canvas JSON State	Rendered Canvas	Boards DB
4.0 Real-Time Sync	Drawing Events	Broadcast to peers	Pusher Channel
5.0 Export/Share	Canvas Data	PNG/PDF/URL	—

Level 2 DFD — User Authentication (Process 1.0)

This diagram expands the authentication process into its sub-steps:



Use Case Summary

Use Case	Actor	Description
UC-01: Register	User	Create a new account with email and password
UC-02: Login	User	Authenticate and access the dashboard
UC-03: Create Board	User	Create a new whiteboard session
UC-04: Draw on Canvas	User	Use tools to draw on the whiteboard
UC-05: Collaborate	User + Collaborator	Multiple users draw on the same board in real time
UC-06: Save Board	System (auto)	Automatically save canvas state to database
UC-07: Share Board	User	Generate and share a link to the board
UC-08: Export Board	User	Download the canvas as PNG or PDF
UC-09: Delete Board	User	Permanently remove a whiteboard

5. GitHub Link

The complete source code of the Virtual Whiteboard project is available on GitHub:

GitHub Repository: https://github.com/Shandilya009/virtual-whiteboard

Repository Details	Information
Repository Name	virtual-whiteboard-laravel
Visibility	Public
Branch	main
Tech Stack	PHP, Laravel, MySQL, Pusher, Bootstrap 5

A. Appendices

Appendices provide additional helpful information supplementing the main SRS content.

A.1 Appendix 1 — ER Diagram Description

The database schema for the Virtual Whiteboard consists of the following main tables:

Table	Key Columns	Description
users	id, name, email, password, email_verified_at, created_at	Stores registered user accounts
boards	id, user_id, title, slug, is_public, canvas_state (JSON), created_at, updated_at	Stores whiteboard sessions and their canvas state
board_collaborators	id, board_id, user_id, role, joined_at	Tracks users who have access to shared boards
sessions	id, user_id, ip_address, payload, last_activity	Laravel session management